Homework / Lab Report

Hao Yang

March 2, 2026

C2 Time Measurement (10 points)

```
Begin Training, Optimizer SGD (
Parameter Group 0
    dampening: 0
    differentiable: False
    foreach: None
    fused: None
    lr: 0.1
    maximize: False
    momentum: 0.9
    nesterov: False
    weight_decay: 0.0005
), Device cuda
Epoch  | Loss     | Acc (%)  | Data Time  | Train Time | Running Time
-------------------------------------------------------------
1      | 1.9153   | 31.96    | 18.0156    | 15.5913    | 33.9499
```

Figure 1: Time Measurement

C3 I/O Optimization (10 points)

```
[hy3169@b-31-10 HPML_lab2]$ ./run_lab2_a.sh --best_worker --cuda
uv is already installed. Skipping installation.
Using PATH = /home/hy3169/.local/bin:/opt/go/1.20.3/bin:/opt/go/1.20.3/bin:/share/apps/pyenv/py3.9/bin:/share/apps/pyenv/py3.9/co
.3/bin:/usr/local/bin:/usr/bin:/usr/local/sbin:/usr/sbin:/opt/slurm/bin
Running lab2...
/scratch/hy3169/HPML_lab2/HPML_lab2/.venv/lib/python3.14/site-packages/torchvision/datasets/cifar.py:83: VisibleDeprecationWarnin
Python or NumPy boolean but got `align=0`. Did you mean to pass a tuple to create a subarray type? (Deprecated NumPy 2.4)
  entry = pickle.load(f, encoding="latin1")
num_work=0, time=17.40100606899989
num_work=4, time=0.9370699059996923
num_work=8, time=1.7759844770001791
num_work=12, time=2.449564083000496
best_work=4, time=0.9370699059996923

Graph has been saved as 'dataloader_benchmark.png'
```

Figure 2: Best Worker Out

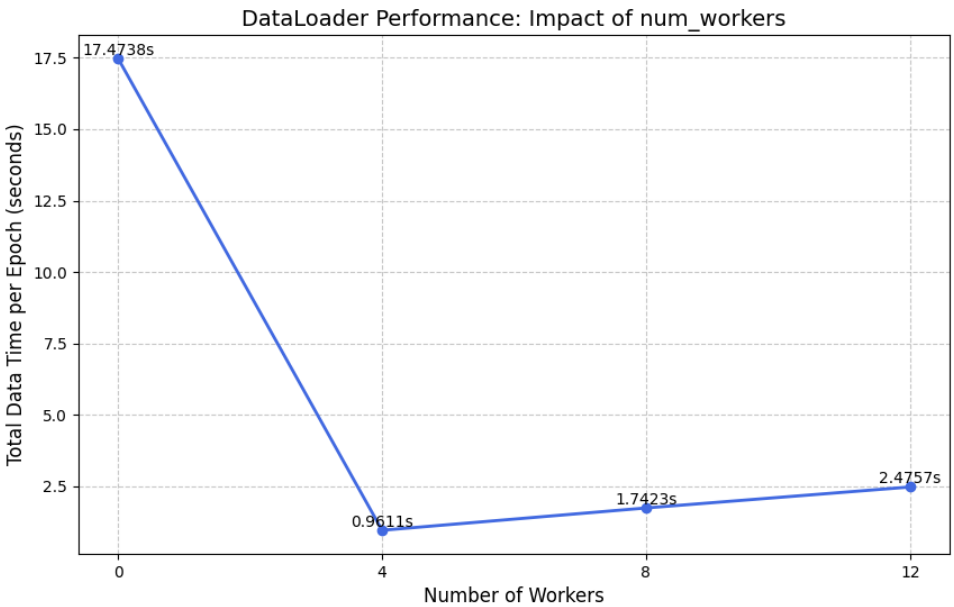

Figure 3: Dataloader Benchmark

C4 Training on GPUs vs CPUs (10 points)

```
Begin Training, Optimizer SGD (
Parameter Group 0
    dampening: 0
    differentiable: False
    foreach: None
    fused: None
    lr: 0.1
    maximize: False
    momentum: 0.9
    nesterov: False
    weight_decay: 0.0005
), Device cuda
Epoch  | Loss     | Acc (%)  | Data Time  | Train Time | Running Time
-------------------------------------------------------------
1      | 1.9666   | 30.88    | 5.0606     | 17.1483    | 22.5627
2      | 1.4984   | 44.59    | 0.9463     | 17.1518    | 18.4373
3      | 1.2370   | 55.44    | 0.9285     | 17.3614    | 18.6335
4      | 1.0053   | 64.25    | 0.9588     | 17.2600    | 18.5562
5      | 0.8378   | 70.46    | 1.0017     | 17.1289    | 18.4727
```

Figure 4: Training on GPUs

```
Begin Training, Optimizer SGD (
Parameter Group 0
    dampening: 0
    differentiable: False
    foreach: None
    fused: None
    lr: 0.1
    maximize: False
    momentum: 0.9
    nesterov: False
    weight_decay: 0.0005
), Device cpu
Epoch  | Loss     | Acc (%)  | Data Time  | Train Time | Running Time
-------------------------------------------------------------
1      | 1.8364   | 33.70    | 5.6590     | 160.4139   | 166.3912
2      | 1.3202   | 51.58    | 1.4077     | 150.2976   | 152.0235
3      | 1.0568   | 62.15    | 1.3736     | 147.8902   | 149.5456
4      | 0.8887   | 68.42    | 1.3940     | 156.6693   | 158.3736
5      | 0.7606   | 73.28    | 1.4351     | 148.8558   | 150.6307
```

Figure 5: Training on CPUs

C5 Experimenting with Different Optimizers (10 points)

```
Begin Training, Optimizer SGD (
Parameter Group 0
    dampening: 0
    differentiable: False
    foreach: None
    fused: None
    lr: 0.1
    maximize: False
    momentum: 0.9
    nesterov: False
    weight_decay: 0.0005
), Device cuda
Epoch  | Loss     | Acc (%)  | Data Time  | Train Time | Running Time
-------------------------------------------------------------
1      | 1.9067   | 31.75    | 5.0043     | 16.9720    | 22.3107
2      | 1.3777   | 49.55    | 0.9395     | 16.9822    | 18.2431
3      | 1.0933   | 61.05    | 0.9510     | 17.1666    | 18.4376
4      | 0.9085   | 67.78    | 0.9375     | 17.3513    | 18.6172
5      | 0.7692   | 73.04    | 0.9298     | 17.2405    | 18.4859
```

Figure 6: SGD

```
Begin Training, Optimizer SGD (
Parameter Group 0
    dampening: 0
    differentiable: False
    foreach: None
    fused: None
    lr: 0.1
    maximize: False
    momentum: 0.9
    nesterov: True
    weight_decay: 0.0005
), Device cuda
Epoch  | Loss     | Acc (%)  | Data Time  | Train Time | Running Time
-------------------------------------------------------------
1      | 1.8454   | 34.07    | 5.4530     | 17.3687    | 23.2121
2      | 1.3018   | 52.40    | 0.9574     | 17.3968    | 18.7441
3      | 1.0202   | 63.76    | 0.9572     | 17.5687    | 18.8857
4      | 0.8260   | 70.93    | 0.9614     | 17.4643    | 18.8035
5      | 0.6957   | 75.80    | 0.9517     | 17.3138    | 18.6351
```

Figure 7: Training on CPUs

```
Begin Training, Optimizer Adam (
Parameter Group 0
    amsgrad: False
    betas: (0.9, 0.999)
    capturable: False
    decoupled_weight_decay: False
    differentiable: False
    eps: 1e-08
    foreach: None
    fused: None
    lr: 0.1
    maximize: False
    weight_decay: 0.0005
), Device cuda
Epoch  | Loss     | Acc (%)  | Data Time  | Train Time | Running Time
-------------------------------------------------------------
1      | 2.1451   | 23.75    | 5.2002     | 17.7518    | 23.3372
2      | 1.8576   | 29.22    | 0.9444     | 17.3462    | 18.6388
3      | 1.8276   | 30.15    | 0.9552     | 17.3187    | 18.6109
4      | 1.8241   | 30.21    | 0.9350     | 17.1541    | 18.4023
5      | 1.8341   | 29.69    | 0.9428     | 16.9865    | 18.2436
```

Figure 8: Adam

C6 Experimenting without Batch Norm (5 points)

```
----------------------------------------------------------------
        Layer (type)               Output Shape         Param #
================================================================
            Conv2d-1           [-1, 64, 32, 32]           1,728
            Conv2d-2           [-1, 64, 32, 32]          36,864
            Conv2d-3           [-1, 64, 32, 32]          36,864
 ResidualBlockNoBN-4           [-1, 64, 32, 32]               0
            Conv2d-5           [-1, 64, 32, 32]          36,864
            Conv2d-6           [-1, 64, 32, 32]          36,864
 ResidualBlockNoBN-7           [-1, 64, 32, 32]               0
            Conv2d-8          [-1, 128, 16, 16]           8,192
            Conv2d-9          [-1, 128, 16, 16]          73,728
           Conv2d-10          [-1, 128, 16, 16]         147,456
ResidualBlockNoBN-11          [-1, 128, 16, 16]               0
           Conv2d-12          [-1, 128, 16, 16]         147,456
           Conv2d-13          [-1, 128, 16, 16]         147,456
ResidualBlockNoBN-14          [-1, 128, 16, 16]               0
           Conv2d-15            [-1, 256, 8, 8]          32,768
           Conv2d-16            [-1, 256, 8, 8]         294,912
           Conv2d-17            [-1, 256, 8, 8]         589,824
ResidualBlockNoBN-18            [-1, 256, 8, 8]               0
           Conv2d-19            [-1, 256, 8, 8]         589,824
           Conv2d-20            [-1, 256, 8, 8]         589,824
ResidualBlockNoBN-21            [-1, 256, 8, 8]               0
           Conv2d-22            [-1, 512, 4, 4]         131,072
           Conv2d-23            [-1, 512, 4, 4]       1,179,648
           Conv2d-24            [-1, 512, 4, 4]       2,359,296
ResidualBlockNoBN-25            [-1, 512, 4, 4]               0
           Conv2d-26            [-1, 512, 4, 4]       2,359,296
           Conv2d-27            [-1, 512, 4, 4]       2,359,296
ResidualBlockNoBN-28            [-1, 512, 4, 4]               0
AdaptiveAvgPool2d-29            [-1, 512, 1, 1]               0
           Linear-30                  [-1, 10]           5,130
================================================================
Total params: 11,164,362
Trainable params: 11,164,362
Non-trainable params: 0
----------------------------------------------------------------
Input size (MB): 0.01
Forward/backward pass size (MB): 6.57
Params size (MB): 42.59
Estimated Total Size (MB): 49.17
----------------------------------------------------------------

Begin Training, Optimizer SGD (
Parameter Group 0
    dampening: 0
    differentiable: False
    foreach: None
    fused: None
    lr: 0.1
    maximize: False
    momentum: 0.9
    nesterov: False
    weight_decay: 0.0005
), Device cuda
Epoch  | Loss     | Acc (%)  | Data Time  | Train Time | Running Time
-------------------------------------------------------------
1      | 2.1322   | 18.67    | 5.0407     | 14.2470    | 19.6051
2      | 1.6961   | 37.16    | 0.9223     | 14.1548    | 15.3718
3      | 1.4831   | 45.72    | 0.9192     | 14.2955    | 15.5184
4      | 1.3006   | 52.98    | 0.9313     | 14.3295    | 15.5568
5      | 1.1323   | 60.13    | 0.9332     | 14.2029    | 15.4331
```

Figure 9: SGD Without Batch Norm

C7 Convert to TorchScript (5 points)

```
2      | 1.4893     | 45.10      | 0.9690       | 17.5498      | 18.8924
Using scripting --> New best acc: 45.10%, JIT model saved to best_model_jit.pt
3      | 1.2317     | 55.22      | 0.9783       | 17.5131      | 18.8781
Using scripting --> New best acc: 55.22%, JIT model saved to best_model_jit.pt
4      | 1.0285     | 63.37      | 0.9873       | 17.3671      | 18.7441
Using scripting --> New best acc: 63.37%, JIT model saved to best_model_jit.pt
5      | 0.8879     | 68.45      | 0.9617       | 17.3024      | 18.6361
Using scripting --> New best acc: 68.45%, JIT model saved to best_model_jit.pt
6      | 0.7843     | 72.31      | 0.9586       | 17.3615      | 18.7049
Using scripting --> New best acc: 72.31%, JIT model saved to best_model_jit.pt
7      | 0.6945     | 75.58      | 0.9634       | 17.4092      | 18.7697
Using scripting --> New best acc: 75.58%, JIT model saved to best_model_jit.pt
8      | 0.6264     | 78.33      | 0.9785       | 17.3816      | 18.7443
Using scripting --> New best acc: 78.33%, JIT model saved to best_model_jit.pt
9      | 0.5847     | 79.75      | 0.9703       | 17.3588      | 18.7165
Using scripting --> New best acc: 79.75%, JIT model saved to best_model_jit.pt
10     | 0.5455     | 81.12      | 0.9696       | 17.3318      | 18.6597
Using scripting --> New best acc: 81.12%, JIT model saved to best_model_jit.pt
[hy3169@b-31-10 HPML_lab2]$ ./run_lab2_b.sh --cuda --eval
uv is already installed. Skipping installation.
Using PATH = /home/hy3169/.local/bin:/opt/go/1.20.3/bin:/opt/go/1.20.3/bin:/share/apps/pyenv/py3.9/bin:/share/apps/pyenv
/bin:/opt/go/1.20.3/bin:/usr/local/bin:/usr/bin:/usr/local/sbin:/usr/sbin:/opt/slurm/bin
Running lab2_torchscript...
============================================================
✓ Successfully loaded TorchScript model from: best_model_jit.pt
✓ Evaluation device: cuda
============================================================
```

Figure 10: Covert to TorchScript

C8 Print Model Graph (5 points)

```
Using scripting --> New best acc: 35.92%, JIT model saved to best_model_jit.pt
==== TorchScript Graph ====
def forward(self,
    x: Tensor) -> Tensor:
  bn1 = self.bn1
  conv1 = self.conv1
  x0 = __torch__.torch.nn.functional.relu((bn1).forward((conv1).forward(x, ), ), False, )
  layer1 = self.layer1
  x1 = (layer1).forward(x0, )
  layer2 = self.layer2
  x2 = (layer2).forward(x1, )
  layer3 = self.layer3
  x3 = (layer3).forward(x2, )
  layer4 = self.layer4
  x4 = (layer4).forward(x3, )
  avgpool = self.avgpool
  x5 = (avgpool).forward(x4, )
  x6 = torch.flatten(x5, 1)
  fc = self.fc
  return (fc).forward(x6, )
```

Figure 11: Model Graph

C9 Evaluate the TorchScript Model (5 points)

```
[hy3169@b-31-10 HPML_lab2]$ ./run_lab2_b.sh --cuda --eval
uv is already installed. Skipping installation.
Using PATH = /home/hy3169/.local/bin:/opt/go/1.20.3/bin:/opt/go/1.20.3/bin:/share/apps/pyenv/py3.9/bin:/share/apps/pyenv/py3.9/condabin:/opt/go/1.
/bin:/opt/go/1.20.3/bin:/usr/local/bin:/usr/bin:/usr/local/sbin:/usr/sbin:/opt/slurm/bin
Running lab2_torchscript...
============================================================
✓ Successfully loaded TorchScript model from: best_model_jit.pt
✓ Evaluation device: cuda
============================================================
/scratch/hy3169/HPML_lab2/HPML_lab2/.venv/lib/python3.14/site-packages/torchvision/datasets/cifar.py:83: VisibleDeprecationWarning: dtype(): align
uld be passed as Python or NumPy boolean but got `align=0`. Did you mean to pass a tuple to create a subarray type? (Deprecated NumPy 2.4)
  entry = pickle.load(f, encoding="latin1")
Accuracy: 74.11%
```

Figure 12: Evaluate the TorchScript Model

C10 Latency Comparsion (5 points)

```
[hy3169@b-31-118 HPML_lab2]$ ./run_lab2_b.sh --latency
uv is already installed. Skipping installation.
Using PATH = /home/hy3169/.local/bin:/opt/go/1.20.3/bin:/opt/go/1.20.3/bin:/share/apps/pyenv/py3.9/bin:/share/apps/pyenv/py3.9/co
.3/bin:/usr/local/bin:/usr/bin:/usr/local/sbin:/usr/sbin:/opt/slurm/bin
Running lab2_torchscript...
PyTorch latency on CPU: 7.82 ms
TorchScript latency on CPU: 6.33 ms
Speedup: 1.23x
```

Figure 13: Latency on CPU (ms)

```
[hy3169@b-31-118 HPML_lab2]$ ./run_lab2_b.sh --latency --cuda
uv is already installed. Skipping installation.
Using PATH = /home/hy3169/.local/bin:/opt/go/1.20.3/bin:/opt/go/1.20.3/bin:/share/apps/pyenv/py3.9/bin:/share/apps/pyenv/py3.9/co
.3/bin:/usr/local/bin:/usr/bin:/usr/local/sbin:/usr/sbin:/opt/slurm/bin
Running lab2_torchscript...
PyTorch latency on GPU: 2.60 ms
TorchScript latency on GPU: 1.69 ms
Speedup: 1.54x
[hy3169@b-31-118 HPML_lab2]$
```

Figure 14: Latency on GPU (ms)

	Latency on CPU (ms)	Latency on GPU (ms)
PyTorch	7.82	2.60
TorchScript	6.33	1.69

CPU Speedup (PyTorch / TorchScript): 1.24× GPU Speedup (PyTorch / TorchScript): 1.54×

Figure 15: Latency Table

Extra Credit Run The model in C++ using LibTorch(10 points)

```
[hy3169@b-31-111 HPML_lab2-1.0.0]$ ls
CMakeLists.txt  __pycache__        build  dataloader_benchmark.png  lab2_data.py         load_model.cpp  model_jit.py    run_cpp_load.sh  run_lab2_b.sh  utils.py
README.md       best_model_jit.pt  data   lab2.py                   lab2_torchscript.py  model.py        pyproject.toml  run_lab2_a.sh    train.py       uv.lock
[hy3169@b-31-111 HPML_lab2-1.0.0]$ cd build/
[hy3169@b-31-111 build]$ ls
CMakeCache.txt  CMakeFiles  Makefile  cmake_install.cmake  resnet_18
[hy3169@b-31-111 build]$ ./resnet_18 ../best_model_jit.pt
Model Loaded Successfully
 1.1150 -3.0054 -0.4461  1.6204 -0.2754
[ CPUFloatType{1,5} ]
```

Figure 16: C++ Load Model

Q1 (4 points)

How many convolutional layers are in the ResNet-18 model?

```
----------------------------------------------------------------
        Layer (type)               Output Shape         Param #
================================================================
            Conv2d-1            [-1, 64, 32, 32]           1,728
       BatchNorm2d-2            [-1, 64, 32, 32]             128
            Conv2d-3            [-1, 64, 32, 32]          36,864
       BatchNorm2d-4            [-1, 64, 32, 32]             128
            Conv2d-5            [-1, 64, 32, 32]          36,864
       BatchNorm2d-6            [-1, 64, 32, 32]             128
     ResidualBlock-7            [-1, 64, 32, 32]               0
            Conv2d-8            [-1, 64, 32, 32]          36,864
       BatchNorm2d-9            [-1, 64, 32, 32]             128
           Conv2d-10            [-1, 64, 32, 32]          36,864
      BatchNorm2d-11            [-1, 64, 32, 32]             128
    ResidualBlock-12            [-1, 64, 32, 32]               0
           Conv2d-13           [-1, 128, 16, 16]           8,192
      BatchNorm2d-14           [-1, 128, 16, 16]             256
           Conv2d-15           [-1, 128, 16, 16]          73,728
      BatchNorm2d-16           [-1, 128, 16, 16]             256
           Conv2d-17           [-1, 128, 16, 16]         147,456
      BatchNorm2d-18           [-1, 128, 16, 16]             256
    ResidualBlock-19           [-1, 128, 16, 16]               0
           Conv2d-20           [-1, 128, 16, 16]         147,456
      BatchNorm2d-21           [-1, 128, 16, 16]             256
           Conv2d-22           [-1, 128, 16, 16]         147,456
      BatchNorm2d-23           [-1, 128, 16, 16]             256
    ResidualBlock-24           [-1, 128, 16, 16]               0
           Conv2d-25             [-1, 256, 8, 8]          32,768
      BatchNorm2d-26             [-1, 256, 8, 8]             512
           Conv2d-27             [-1, 256, 8, 8]         294,912
      BatchNorm2d-28             [-1, 256, 8, 8]             512
           Conv2d-29             [-1, 256, 8, 8]         589,824
      BatchNorm2d-30             [-1, 256, 8, 8]             512
    ResidualBlock-31             [-1, 256, 8, 8]               0
           Conv2d-32             [-1, 256, 8, 8]         589,824
      BatchNorm2d-33             [-1, 256, 8, 8]             512
           Conv2d-34             [-1, 256, 8, 8]         589,824
      BatchNorm2d-35             [-1, 256, 8, 8]             512
    ResidualBlock-36             [-1, 256, 8, 8]               0
           Conv2d-37             [-1, 512, 4, 4]         131,072
      BatchNorm2d-38             [-1, 512, 4, 4]           1,024
           Conv2d-39             [-1, 512, 4, 4]       1,179,648
      BatchNorm2d-40             [-1, 512, 4, 4]           1,024
           Conv2d-41             [-1, 512, 4, 4]       2,359,296
      BatchNorm2d-42             [-1, 512, 4, 4]           1,024
    ResidualBlock-43             [-1, 512, 4, 4]               0
           Conv2d-44             [-1, 512, 4, 4]       2,359,296
      BatchNorm2d-45             [-1, 512, 4, 4]           1,024
           Conv2d-46             [-1, 512, 4, 4]       2,359,296
      BatchNorm2d-47             [-1, 512, 4, 4]           1,024
    ResidualBlock-48             [-1, 512, 4, 4]               0
AdaptiveAvgPool2d-49             [-1, 512, 1, 1]               0
           Linear-50                   [-1, 10]           5,130
================================================================
Total params: 11,173,962
Trainable params: 11,173,962
Non-trainable params: 0
----------------------------------------------------------------
Input size (MB): 0.01
Forward/backward pass size (MB): 11.25
Params size (MB): 42.63
Estimated Total Size (MB): 53.89
----------------------------------------------------------------
```

Figure 17: ResNet-18 architecture

As shown in Figure 17, **ResNet-18 contains 17 convolutional layers**. It consists of one initial convolution layer, followed by four residual stages. Each stage has two BasicBlocks, and each block contains two convolutional layers:

$$1 + (4 \text{ stages}) \times (2 \text{ blocks}) \times (2 \text{ convs}) = 17.$$

Q2 (4 points)

What is the input dimension of the last linear layer?

The final fully-connected layer of ResNet-18 receives a **512-dimensional input vector**, produced by global average pooling after the last residual block.

Q3 (8 points)

How many trainable parameters and how many gradient elements are in the ResNet-18 model you built (using the SGD optimizer)?

Trainable Parameters

The code used to compute the number of trainable parameters is shown below:

```
from torchsummary import summary
model = ResNet18(num_classes=10).to(device)
summary(model, input_size=INPUT_SIZE, device=str(device))
```

The output indicates that the model contains a total of **11,173,962 trainable parameters**.

Gradient Count

To compute the number of gradient elements, the following code was used:

```
if not printed:
    total_grads = sum(
        p.grad.numel() for p in model.parameters()
        if p.requires_grad and p.grad is not None
    )
    print(f"Gradient elements: {total_grads}")
    printed = True
```

```
Begin Training, Optimizer SGD (
Parameter Group 0
    dampening: 0
    differentiable: False
    foreach: None
    fused: None
    lr: 0.1
    maximize: False
    momentum: 0.9
    nesterov: False
    weight_decay: 0.0005
), Device cpu
Epoch  | Loss     | Acc (%)  | Data Time  | Train Time
-----------------------------------------------------------
[INFO] DataLoader workers: 4
Training:   0%|                                                                                                  | 0/391 [00:00<?, ?batch
Gradient elements: 11173962
```

Figure 18: Number of gradient elements using SGD.

As shown in Figure 18, the number of gradient elements is **exactly equal to the number of trainable parameters**: **11,173,962**.

Q4 (4 points)

Same question as Q3, but using the Adam optimizer (only the answer is required).

Changing the optimizer does not modify the architecture of the model or the number of parameters. Therefore, when using the Adam optimizer, the total number of trainable parameters is still **11,173,962**, and the number of gradient elements remains the same.

```
Begin Training, Optimizer Adam (
Parameter Group 0
    amsgrad: False
    betas: (0.9, 0.999)
    capturable: False
    decoupled_weight_decay: False
    differentiable: False
    eps: 1e-08
    foreach: None
    fused: None
    lr: 0.1
    maximize: False
    weight_decay: 0.0005
), Device cuda
Epoch  | Loss     | Acc (%)  | Data Time  | Train Time
-------------------------------------------------------------
[INFO] DataLoader workers: 4
Training:   0%|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | 0/391 [00:00<?, ?bat
Gradient elements: 11173962
1      | 2.3342     | 20.01      | 5.2208       | 17.2199
```

Figure 19: Number of gradient elements using Adam.

As shown in Figure 19, the gradient count using Adam remains **11,173,962**.

Q5 (4 points)

Explain the differences between tracing and scripting and how they are used in TorchScript.

Tracing

```
import torch
net = ResNet18()
exmaple_weight = torch.randn(1, 3, 32, 32)
traced = torch.jit.trace(model, example)
```

It runs the model once or more times on example inputs and records the tensor ops that actually executed. Also the if the model has data-dependent control flow like if (x.sum ¿ 0), tracing will only record the brach taken for the example input and may ignore other branches.

Scripting

```
import torch
net = ResNet18()
scripted = torch.jit.script(model)
```

It compiles a subset of PyTorch/Python model code into TorchScript by analyzing the model code. It captures control flows and others like TorchScript IR, which enable it to handle input-dependent branching. But the code must be compatible with TorchScript, some Python features and dynamic patterns aren't supported.

TorchScript: Tracing vs. Scripting

Q6 (4 points)

Explain any changes needed in the ResNet-18 model to allow for scripting.

In my repository, I provide two versions of the model: `model.py` (standard PyTorch) and `model_jit.py` (TorchScript-friendly). To make ResNet-18 compatible with `torch.jit.script`, I made the following changes:

- **Replace third-party tensor reshape with a PyTorch op.** In `model.py`, I used `einops.rearrange` to make the shape change easier to read:

  ```
  x = rearrange(x, 'b c 1 1 -> b c')
  ```

  However, since `einops` is a third-party Python library and may not be TorchScript-scriptable, I replaced it with the built-in PyTorch operator:

  ```
  x = torch.flatten(x, 1)
  ```

  After `AdaptiveAvgPool2d((1,1))`, the tensor has shape `[B, C, 1, 1]`, so `torch.flatten(x, 1)` is equivalent and scriptable.

- **Remove jaxtyping annotations and use TorchScript-supported types.** In `model.py`, the forward method used `jaxtyping` annotations:

  ```
  def forward(self, x: Float[Tensor, "batch in_channels h w"]) -> Float[Tensor, "batch num_c
  ```

  TorchScript expects standard types such as `Tensor`, so in `model_jit.py` I simplified it to:

  ```
  def forward(self, x: Tensor) -> Tensor:
  ```

- **Avoid potentially problematic in-place ops (more conservative for scripting).** In the JIT version, I use `F.relu(out)` instead of `F.relu(out, inplace=True)` to avoid any in-place operation issues during scripting/graph compilation.

- **Simplify Python typing for layer construction.** In `model.py`, I explicitly annotated the Python list type:

  ```
  layers: list[ResidualBlock] = [ResidualBlock(...)]
  ```

  In `model_jit.py`, I removed this annotation and used the JIT block class:

  ```
  layers = [ResidualBlockJIT(...)]
  ```

  This avoids extra Python typing that can interfere with TorchScript compilation.